

```

❹ oBall.draw() # tell the ball to draw itself
    oFrameCountLabel.draw()
    oFrameCountDisplay.draw()
    oRestartButton.draw()

# 11 - Update the window
pygame.display.update()

# 12 - Slow things down a bit
clock.tick(FRAMES_PER_SECOND)

```

Listing 6-7: An example main program to show `Ball`, `SimpleText`, and `SimpleButton`

At the top of the program, we import the code of the `Ball`, `SimpleText`, and `SimpleButton` classes ❶. Before our main loop starts, we create an instance of the `Ball` ❷, two instances of the `SimpleText` class (`oFrameCountLabel` for the unchanging message label and `oFrameCountDisplay` for the changing display of frames), and an instance of the `SimpleButton` class that we store in `oRestartButton`. We also initialize a variable `frameCounter` to zero, which we will increment every time through the main loop.

In the main loop, we check if the user pressed the Restart button ❸. If `True`, we reset the frame counter.

We tell the ball to update its position ❹. We increment the frame counter, then call the `setValue()` method of the text field to show the new count of frames ❺. Finally, we tell the ball to draw itself tell the text fields to draw themselves, and tell the Restart button to draw itself, by calling the `draw()` method of each object ❻.

In the instantiation of the `SimpleText` objects, the last argument is a text color, and we specified that the objects should be rendered in `WHITE` so they can be seen against a `BLACK` background. In the next chapter, I'll show how to expand the `SimpleText` class to incorporate more attributes, without complicating the interface of the class. We'll build a more full-featured text object that has reasonable default values for each of these attributes, but allows you to override those defaults.